

Integrating Bluetooth into the Monkey-Head Project

This technical report evaluates the use of Bluetooth wireless technology within the Monkey-Head Project (MHP). It covers the fundamentals of Classic Bluetooth vs. Bluetooth Low Energy (BLE), short-range performance metrics, device connectivity methods, and how Bluetooth could be applied to MHP's hardware and workflows. We also propose a design for a Bluetooth-based redundant file transfer layer, and examine security implications. All information is up-to-date as of mid-2025, drawing on official Bluetooth SIG documentation and reputable technical sources.

1. Overview of Bluetooth Protocols (Classic vs. BLE)

Classic Bluetooth (BR/EDR): "Classic" Bluetooth refers to the original specification (Bluetooth versions 1.x to 3.x) encompassing Basic Rate and Enhanced Data Rate modes ¹. It operates in the 2.4 GHz ISM band with 79 channels and was designed for continuous, high-throughput connections such as audio streaming or file exchange ² ³. Classic Bluetooth forms point-to-point links (a **piconet** architecture with one master and up to 7 active slaves) to create personal area networks ⁴ ⁵. This protocol excels at supporting legacy devices and high data-rate applications (e.g. wireless headphones, file transfer, classic HID devices) but at the cost of higher power consumption ⁶ ⁷.

Bluetooth Low Energy (BLE): Introduced in Bluetooth 4.0, BLE (also known as Bluetooth Smart) is a distinct, incompatible protocol optimized for low power operation ⁸ ⁹. BLE also uses 2.4 GHz but with 40 channels and a different frequency-hopping scheme ¹⁰. It was designed for devices that send small amounts of data infrequently, enabling coin-cell battery operation for months or years ¹¹ ¹². BLE connections incur very short active periods and can remain in sleep mode most of the time, drastically reducing average power draw ¹³. In terms of topology, BLE is more flexible: it supports one-to-one communication like Classic, but also one-to-many (broadcast mode) and many-to-many via mesh networking ¹⁴ ⁴. This makes BLE well-suited for IoT sensor networks, beacons, and other applications where energy efficiency and scalability matter more than raw throughput ¹⁵.

Protocol Evolution: Both Classic and BLE continue to evolve under the Bluetooth SIG. Modern Bluetooth 5+ devices often support **dual mode** (both Classic and BLE radios). Bluetooth 5 introduced significant improvements for BLE – doubling the maximum data rate and increasing range – without changing Classic Bluetooth's fundamentals ¹⁶ ¹⁷. However, the two protocols address different use cases: Classic for high-bandwidth continuous streams (audio, large data) and BLE for low-bandwidth intermittent data (sensors, control signals) with minimal power usage ¹⁸ ¹⁹.

2. Bluetooth Short-Range Data Transmission Capabilities

In evaluating Bluetooth for short-range data links in MHP, key performance metrics include **bandwidth**, **latency**, **range**, and **reliability**, especially under real-world environmental conditions. Below we summarize these characteristics for Classic Bluetooth and BLE:

2.1 Bandwidth and Data Rates

- **Classic Bluetooth Throughput:** Classic BR/EDR supports a theoretical maximum gross air data rate of 3 Mb/s (with Enhanced Data Rate), which yields roughly up to ~2 Mb/s of practical throughput for user data ²⁰. In practice, real file transfer speeds over Classic Bluetooth (e.g. via the Serial Port Profile or OBEX file transfer) tend to be on the order of a few hundred kilobytes per second. This higher throughput enables use cases like audio streaming and bulk file transfers that would be impractically slow over lower-bandwidth links ²¹.
- **BLE Throughput:** BLE was initially limited to 1 Mb/s physical bit rate, but Bluetooth 5 introduced a 2 Mb/s PHY option for higher speed, as well as a long-range coded PHY (500 kb/s or 125 kb/s) ¹⁶. Even with the 2 Mb/s PHY, protocol overhead (advertising intervals, connection events, acknowledgments) means **effective BLE data throughput** is typically far below the radio bit rate. In Bluetooth 4.x (1 Mb/s), practical data throughput is on the order of 100–250 kb/s ²². With Bluetooth 5's 2 Mb/s mode and optimized parameters, real-world throughput up to ~1 Mb/s (around 0.8 Mb/s or ~100 kB/s) can be achieved under ideal conditions, though ~0.5 Mb/s is more common. **In summary, Classic Bluetooth still exceeds BLE in sustained data rate** (roughly 2–3× higher), making Classic a better choice for large file transfers or high-fidelity streams ²³ ²⁰. BLE's bandwidth, while lower, is usually sufficient for periodic sensor readings, control messages, and other low-data-rate tasks in MHP.
- **Impact on MHP:** For the Monkey-Head Project, this means that using Classic Bluetooth would allow faster bulk data transfers (e.g. sending firmware updates or large log files) compared to BLE. On the other hand, BLE's throughput is adequate for telemetry or command/control data, and its lower data rate comes with lower power usage. If the requirement is to occasionally send multi-megabyte files as a redundancy measure, Classic Bluetooth (or the Bluetooth Classic mode of a dual-mode module) may be preferable to meet time constraints.

2.2 Communication Latency

- **Classic Bluetooth Latency:** Classic connections have relatively higher latency, on the order of ~100 ms for typical link setups ²⁴. The base Bluetooth BR/EDR physical layer uses time slots of 625 μ s and a polling scheme; a device may take tens of milliseconds to respond depending on scheduling and any retransmissions. In practical terms, human interactions like audio have ~100 ms latency over Classic profiles, and simple data exchange pings are often in the few tens of milliseconds at best. Classic's latency, while low enough for audio and many control tasks, is not as low as BLE's in optimized cases.
- **BLE Latency:** BLE was designed for low latency. It supports connection intervals as short as 7.5 ms, meaning a peripheral can send data to a central with latency on the order of 10 ms or less in ideal conditions ²⁴. In practice, BLE can achieve ~6–20 ms latency for a data packet from one device to

another ²⁵. This responsiveness is a key advantage for real-time monitoring and control. For example, a BLE-based sensor can transmit readings or an actuator can receive a command with only a few milliseconds of delay, which is critical for time-sensitive operations.

- **Latency Considerations for MHP:** Low latency is important if Bluetooth is used for **control signals or real-time feedback** in the Monkey-Head Project. BLE's sub-10 ms latency can support fast control loops or quick status updates (e.g. emergency stop commands, live sensor streams) with minimal lag. Classic Bluetooth's ~100 ms latency might be noticeable in tightly coupled control but is still acceptable for non-critical commands or file transfers where a fraction of a second delay is not an issue. Overall, **BLE is more suitable for applications demanding responsiveness**, whereas Classic is sufficient for moderate-speed interactions and data transfer that is not latency-critical.

2.3 Range and Signal Propagation

- **Typical Range:** Bluetooth is inherently a short-range wireless technology. Classic Bluetooth devices (Class 2 radios, ~2.5 mW) have a reliable range of roughly 10 meters indoors, often up to ~30 m in open space ¹⁷. BLE devices in practice achieve slightly longer range in open air – about 50 m reliably, and up to ~150 m line-of-sight under ideal outdoor conditions with Bluetooth 5 long-range features ¹⁷. Notably, Bluetooth 5's coded PHY (125 kbps mode) enables **extended range (4×)** at the cost of throughput, and high-power Class 1 Bluetooth radios (100 mW) can approach 100 m range in open environments.
- **Environmental Factors:** Real-world range is highly dependent on environmental conditions. The 2.4 GHz signals are **strongly attenuated by obstacles** such as walls, metal surfaces, and even the human body (water content) ²⁶. Indoors, Bluetooth range may drop to a single room or adjacent room if dense walls or metal equipment are present. Multipath fading and interference can also limit practical range and reliability. Antenna design and placement are critical: a well-designed antenna in the Monkey-Head Project's hardware, mounted with clear line-of-sight, can maximize range, whereas an internal or obstructed antenna may reduce it ²⁷. Device orientation and enclosure materials (for example, a metal chassis) will likewise impact the signal propagation ²⁷. In summary, achieving the high-end of Bluetooth range (tens of meters) requires optimal conditions – open air, minimal RF noise, and good antenna performance.
- **Interference and Reliability:** Bluetooth shares the 2.4 GHz ISM band with Wi-Fi, microwaves, and other devices, so **interference** is a key consideration. Both Classic and BLE use frequency hopping spread spectrum (FHSS) across their channels to mitigate interference; in fact, Bluetooth adaptively hops 1600 times per second and can blacklist crowded channels (Adaptive Frequency Hopping) to avoid Wi-Fi bands ²⁸. This gives Bluetooth decent robustness in noisy environments. However, under heavy Wi-Fi traffic or RF noise, Bluetooth throughput can degrade and latency can increase due to packet loss and retries. From a reliability standpoint, Bluetooth connections are generally stable at short range, but in the **worst-case environment** (e.g. an industrial area with lots of 2.4 GHz noise, or if the Monkey-Head robot is surrounded by multiple active Wi-Fi APs), throughput might fall well below peak values and occasional dropouts could occur. Careful testing in the intended deployment environment is thus recommended.
- **Range/Environment in MHP:** The Monkey-Head Project should expect solid Bluetooth communication within the same room or a radius of several meters, which is appropriate for most

on-robot or near-robot use cases (like a technician standing next to the robot using a Bluetooth device). For multi-room coverage or outdoor use beyond ~30–50 m, Bluetooth would be unreliable without repeaters or a mesh network. Given Huey's physical shell is a high-end server chassis, integrating a quality external antenna or using a dongle on a USB extension may be wise to avoid the metal enclosure blocking the signal. Overall, Bluetooth's range is sufficient for **short-range localized data exchange** and on-site wireless control, but it is not meant to replace long-range wireless links (like Wi-Fi or cellular) if those are needed for remote operation.

2.4 Reliability Under Various Conditions

- **Line-of-Sight vs. Obstructions:** In line-of-sight open conditions, both Classic and BLE can maintain very reliable links across the specified ranges (tens of meters). With obstructions, reliability diminishes. For example, a concrete wall might reduce the signal strength by an order of magnitude, shortening reliable range to a few meters. Human bodies or crowds around the devices can absorb signal (since 2.4 GHz is absorbed by water), causing variability if people move around ²⁶. In the context of MHP, if the robot's Bluetooth link is to a wearable or handheld device, simply turning or having the robot's body between the two devices could momentarily affect the signal strength. Planning antenna placement (e.g. possibly mounting a Bluetooth antenna on the robot's exterior or head) can mitigate these issues.
- **Adaptive Features:** Modern Bluetooth implementations have features to enhance link reliability. Both Classic and BLE perform **power control** – devices can adjust transmit power to the minimum needed, and increase it if error rates rise (within their class limits). BLE's coded PHY uses error-correcting codes to successfully deliver packets at longer distances or in noisy environments at the expense of speed ²⁹. For critical data, BLE also offers an *Acknowledged* mode (for GATT writes or by using notifications with application-layer ack) to ensure data receipt. These features mean that even under suboptimal conditions, Bluetooth will often *throttle down* (either by reducing speed or by retransmitting) rather than completely dropping the connection. From a reliability standpoint, this translates to graceful degradation under interference: the data will get through, but latency or throughput might suffer.
- **Reliability for MHP:** The key takeaway for the Monkey-Head Project is that Bluetooth can provide a **reliable short-range link for data and control** as long as its limitations are respected. It should be treated as a *local wireless extension* of the system, with expected variability in throughput. For non-critical telemetry and backup data transfer, occasional slow-downs due to interference are acceptable. If using Bluetooth for any safety-critical control (e.g. an emergency stop signal), it's important to test that signal gets through quickly even in worst-case RF conditions (or use redundancy, such as sending repeated stop commands). Generally, the fast reconnect times and robust error handling of BLE make it suitable for IoT-style reliability (devices can disconnect and reconnect within seconds or less), which aligns with intermittent use in a redundant system.

3. Interconnectivity Options and Multi-Device Networking

Beyond basic point-to-point links, Bluetooth offers various ways to connect multiple devices and integrate into complex systems. We examine how devices pair, how many connections can be supported, advanced network topologies like mesh, and how Bluetooth might interface with MHP's existing hardware and subsystems.

3.1 Pairing Mechanisms and Connection Setup

Classic Bluetooth Pairing: Classic Bluetooth typically requires an explicit pairing process to establish trust between devices. Modern implementations use **Secure Simple Pairing (SSP)** which was introduced in Bluetooth 2.1. SSP utilizes an ECDH (Elliptic Curve Diffie-Hellman) key exchange and supports different association models: *Just Works*, *PIN/passkey entry*, or *Numeric Comparison* on devices with a display ³⁰ ³¹. In practice, for devices without a UI (like many embedded modules), a fixed PIN or Just Works pairing is used, which is convenient but offers no man-in-the-middle protection. Once paired, devices generate a link key and enable encryption for all communications. The pairing is usually one-time; afterwards, devices “bond” and recognize each other to reconnect without re-pairing. Classic’s pairing process is user-centric (often requiring approval or a PIN), which suits peripheral accessories. In MHP, if Classic Bluetooth is used (e.g. via a USB Bluetooth dongle on Huey), we’d likely configure a fixed PIN or use Just Works for ease of pairing a headless robot with a laptop/phone. Alternatively, implementing Numeric Comparison (displaying a code on the connected computer and confirming on a client) would provide stronger authentication if a user interface is available during pairing.

BLE Connection and Pairing: BLE offers more flexibility – devices can communicate in two modes: - **Unpaired, broadcast mode:** A BLE peripheral can send advertising packets to any scanner without needing pairing. This is useful for one-way data like beacons, simple sensor broadcasts, or discovery of devices. - **Paired (bonded) connections:** For two-way data exchange or secure data, BLE devices go through a pairing process to exchange keys and enable encryption. BLE’s pairing in modern Bluetooth 5+ devices uses **LE Secure Connections** (also ECDH-based, similar in concept to SSP) to derive a 128-bit Long Term Key for encryption ³² ³¹. BLE pairing has association models analogous to Classic: Just Works, Passkey, Numeric Comparison, or Out-of-Band. The appropriate method is chosen based on device IO capabilities, and developers are advised to choose the most secure method possible (avoiding unauthenticated Just Works if feasible) ³³ ³⁴.

One notable difference is that **BLE does not strictly require pairing if the data is meant to be open**. For example, a temperature sensor could simply advertise its readings, and any BLE scanner (within range) can receive it without formal bonding. In the context of MHP, this means certain data like non-sensitive telemetry could be broadcast for convenience (no need to pair every time a new diagnostics tool wants to listen). However, any control commands or file transfers would necessitate a connected mode with encryption, implying a pairing step.

Connection Setup Speed: BLE has an edge in how quickly connections can be established. It was designed for fast connect/disconnect cycles; a BLE device can advertise and connect in a fraction of a second. Classic Bluetooth pairing and connection establishment is slower (sometimes several seconds to discover, pair, and open a RFCOMM channel). If MHP envisions connecting on the fly (for instance, a technician approaches the robot with a tablet and quickly links via BLE to download logs), BLE’s quicker handshaking is beneficial ³⁵. On the other hand, maintaining a continuous link (e.g. during a long file transfer) is well within both protocols’ capabilities once the connection is up.

3.2 Multi-Device Support (Piconets, Multiple Connections)

Classic (BR/EDR) Multi-Device: A single Classic Bluetooth master can maintain connections with up to **7 active slave devices** in a piconet ⁵. In theory, by time-sharing, one device could communicate with multiple peripherals (for example, a computer connected to several sensors via SPP simultaneously). In

practice, however, Classic Bluetooth is most often used in one-to-one or one-to-two scenarios (e.g. a phone connected to a headset and maybe a smartwatch concurrently). More complex “scatternet” configurations (a device participating in multiple piconets to extend the network) were defined in the spec but are rarely implemented in consumer devices. Essentially, Classic Bluetooth’s multi-device networking is limited and often sequential (the master polls each slave in turn). For MHP, if we rely on Classic Bluetooth, the system could pair with a handful of devices at once (say, a laptop, a phone, and a sensor module), but bandwidth is shared among them. Each additional active connection will consume time slots and reduce throughput per device, and beyond 7 devices the master cannot maintain more simultaneous links.

BLE Multi-Device: BLE was built with **flexible multi-connection** in mind. A BLE **Central** device (like a hub) can connect to a large number of Peripherals concurrently – the limit is mostly up to the BLE stack implementation and available memory/processing. Dozens of simultaneous BLE connections are possible on modern hardware, and effectively **there isn’t a fixed 7-device limit** ⁵. For example, an IoT gateway might maintain 20 BLE peripheral links at once (e.g. to 20 sensors). Additionally, BLE supports *broadcast* communication: one device can advertise data that unlimited observers could receive, and *observer* devices can listen to multiple broadcasters. This means one-to-many data distribution is inherently supported (unlike Classic, which always pairs).

For the Monkey-Head Project, if multiple subsystems or sensors were to use BLE, a single central hub (e.g. the main robot’s computer with a BLE adapter) could gather data from all of them concurrently. Similarly, the robot could broadcast a status packet and multiple monitoring devices (laptops, phones) could pick it up simultaneously. **Bluetooth Mesh** (covered below) further extends the multi-device story for BLE by enabling beyond-radio-range and many-to-many communications.

3.3 Mesh Networking with BLE

One of the compelling features introduced by the Bluetooth SIG in recent years is the **Bluetooth Mesh** profile (released in 2017). Bluetooth Mesh operates on top of BLE and allows formation of large-scale device networks in a many-to-many topology ¹⁴ ⁴. In a Bluetooth Mesh network, devices (called nodes) relay messages to each other such that devices need not be in direct radio range of every other device; messages hop across intermediate nodes. This is ideal for scenarios like building automation (lighting systems, sensor arrays) where potentially tens or hundreds of devices are spread out.

Mesh Characteristics: Bluetooth Mesh uses a flooding technique (with managed flooding to control message count) and is optimized for relatively low-rate, small payload data (like sensor readings, commands). It is not designed for high-bandwidth transfers; rather, it excels at reliability and scale. The mesh specification supports up to **thousands of nodes (practically 100+ is common, with a theoretical limit around 32,000 nodes)** in one network by using 16-bit addresses for nodes. All communications in mesh are addressed (to groups or single nodes) and are encrypted with network and application keys, ensuring security and message authenticity across the mesh.

Relevance to MHP: If the Monkey-Head Project envisions many distributed devices or modules that need to coordinate via wireless (for instance, separate sensor pods or multiple robots working in concert), Bluetooth Mesh is a candidate. A mesh could allow sensors on different parts of a facility to send data to the robot, or allow multiple robots to share data, without requiring direct point-to-point links for every pair. However, implementing mesh adds complexity – it requires all participating devices to run the Bluetooth Mesh stack and tends to be used for fixed infrastructure (like stationary sensors) due to latency (mesh hops add delay,

usually tens to hundreds of ms). For a single robot unit and a handful of nearby devices, **mesh is likely overkill**. Simple BLE star topology (one central, multiple peripherals) or occasional broadcasts should suffice for MHP's needs. Still, it's good to know that if a multi-hop network is ever needed (e.g. the project expands to a multi-room sensor network), Bluetooth Mesh provides a standards-based way to achieve that on BLE radios.

3.4 Integration with Monkey-Head Project Hardware

The Monkey-Head Project's hardware platform ("Huey" robotic shell) can integrate Bluetooth in several ways. Given Huey's core uses a Supermicro X9 server-class motherboard ³⁶ (which does not include onboard wireless radios), adding Bluetooth will involve external hardware or modules:

- **USB or PCIe Adapter:** The simplest path is to use a Bluetooth 5.x USB dongle on Huey's main computer. These adapters typically support dual-mode (Classic + BLE) and are inexpensive. Driver support on the HostOS (Windows or Linux) is excellent – for example, on Linux the BlueZ stack can manage the adapter, and on Windows the built-in Bluetooth stack would handle it. Using a standard adapter leverages the full Bluetooth stack of the OS, meaning we can utilize high-level APIs to implement profiles (RFCOMM sockets for Classic, GATT APIs for BLE, etc.). For MHP, a USB adapter plugged into Huey and mounted with an external antenna (to avoid the metal case attenuating signals) would likely be the fastest way to get Bluetooth connectivity.
- **Dedicated Bluetooth Module:** Another approach is adding a microcontroller or wireless module with Bluetooth capability and interfacing it to the main system. For instance, an Arduino or STM32 with a Bluetooth (or an SoC like ESP32 that has built-in Bluetooth) could serve as a subordinate device that handles Bluetooth communication and forwards data to the main system via UART, USB, or SPI. Classic modules like the **HC-05** (Classic SPP) or **HC-06** exist, as do BLE modules like **nRF52-based boards**. However, since Huey already runs a full OS, using the OS's stack via a dongle is probably more straightforward than managing an external module's firmware.
- **Integration with Subsystems:** If certain subsystems in MHP (say, a sensor cluster or a wearable controller) already have Bluetooth support, we should ensure compatibility. For example, if the project uses a Raspberry Pi for a SubOS layer (Raspberry Pis have Bluetooth Low Energy built-in), that could directly be leveraged. The "Test Hardware" or any legacy integration mentioned in the project should be surveyed for Bluetooth capability. According to the documentation, MHP emphasizes **legacy hardware integration** (even things like vintage computers), but those likely do not involve Bluetooth. So integration will mainly focus on modern components.
- **Pairing & User Interface:** Integrating Bluetooth means we need a strategy for pairing devices with the robot. On the software side, one could implement a command-line or GUI utility to put the robot in pairing mode or to initiate pairing with a known device. Alternatively, for BLE, the robot can advertise a custom service and accept connections without manual pairing if security is not a concern for some data. Given MHP's focus on ethical and secure design, we should plan to handle pairing in a controlled manner (perhaps a secure pairing with a known passkey for authorized engineer devices, etc.). The integration should also include an indication (LED or log message) when Bluetooth is connected, to aid in debugging and trust.

- **Antenna and Power:** The Bluetooth radio's performance will depend on the antenna. If using a dongle, select one with an external antenna or at least position the dongle outside dense metal areas. Bluetooth is low power, so running it will have negligible impact on Huey's power budget. However, if the robot has multiple wireless systems (e.g. Wi-Fi and Bluetooth running concurrently), ensure there's physical separation of antennas to reduce interference and that the thermal environment for the radio is within spec (unlikely to be an issue, but mention for completeness).

In summary, integrating Bluetooth into the Monkey-Head Project should be technically straightforward. The team will need to decide on the exact hardware (USB adapter vs. integrated module) and then incorporate the Bluetooth stack or libraries into the software. Once in place, this will open up numerous connectivity options as discussed below.

4. Potential Bluetooth Use Cases in the Project

With Bluetooth added to MHP, there are several **use cases** where it could enhance or complement existing capabilities. Below are key scenarios envisaged for Bluetooth within the project:

- **Wireless Sensor Telemetry:** MHP could use BLE to collect data from **wireless sensors** on or around the robot. For example, small BLE sensor nodes (measuring temperature, orientation, battery levels, etc.) attached to moving parts of the robot or in its environment could transmit their readings to GenCore. This would reduce cabling and allow easy addition of new sensors. BLE's support for broadcasting and star networks fits this use – the robot's central system can listen to periodic broadcasts from multiple sensors. As an illustration, many industrial and medical devices use BLE for exactly this purpose – periodic, low-volume telemetry with years of battery life ¹⁵. In MHP, a BLE sensor could broadcast, say, the joint torque reading of an appendage, and the central system could adjust control parameters accordingly in near real-time.
- **Remote Control and Command Signals:** Bluetooth can serve for **direct remote control** of the robot or its subsystems over short range. For instance, an engineer's smartphone or a dedicated Bluetooth remote could pair with the robot to send control commands (movement instructions, mode switches, calibration commands). BLE's low latency (~10 ms) means even interactive control (joystick, gamepad, or gesture control via a BLE wearable) is feasible without perceptible lag ²⁵. Indeed, many hobby and consumer robots are controlled via Bluetooth from phone apps or controllers. Classic Bluetooth could also be used with standard HID profiles to connect a game controller or keyboard for manual control in the field. Using Bluetooth for control has the benefit of not requiring network infrastructure (useful in a lab or workshop, or emergency stop via a nearby link). However, it's best limited to short-range scenarios (line-of-sight, within a few meters) to ensure reliability.
- **Diagnostics and Maintenance Interface:** Bluetooth provides a convenient way to access **diagnostic data** from the robot for maintenance. For example, when a technician is physically near the robot, they could connect via Bluetooth to pull system logs, error reports, or performance metrics without needing a wired link or network connection. The robot could advertise a BLE **diagnostics service** that, when connected, allows reading various status characteristics (temperatures, CPU load, sensor health, etc.). This approach is common in IoT devices – many appliances and machines have a BLE maintenance app that technicians use on-site. In MHP, an advantage is isolation: a Bluetooth maintenance link can be kept local (not exposed over IP),

improving security while the robot is in an environment where network access might be restricted. We could also use Bluetooth to push updates in a pinch (if the main network is down) – e.g., send a small patch or new config file from a phone to the robot via BLE.

- **Real-Time Monitoring HUD:** Building on the above, Bluetooth could enable a **real-time monitoring HUD (Heads-Up Display)** for developers or operators. Imagine a tablet or AR headset that connects via BLE to the robot and streams live telemetry (albeit low-bandwidth) such as battery level, motor currents, or AI status. Because BLE can handle a few dozen kilobytes per second easily, it could stream a continuous feed of critical metrics at, say, 5–10 Hz update rates. This is enough for a human-readable dashboard or even basic plots. Such a tool would be invaluable during field testing of the robot, where the engineer can walk around with the robot and see its internals on a handheld device without tethering. It adds mobility – no wires or fixed console needed – aligning with MHP’s goal of adaptable, user-friendly operation.
- **Multi-Device Coordination:** If multiple robots or units are part of MHP in the future, Bluetooth could assist in their coordination locally. For instance, two robots in proximity could use Bluetooth to quickly exchange state information or sensor data (forming an ad-hoc cluster). This is a more speculative use case, as full multi-robot coordination might rely on Wi-Fi or a dedicated radio for range. But for short-range, high-robustness needs (like collision avoidance signals between robots in a lab), Bluetooth’s frequency hopping and low latency make it a reliable channel for device-to-device alerts.

Overall, Bluetooth’s role in the Monkey-Head Project would likely revolve around **short-range, on-demand interactions**: getting data off the robot when nearby, sending commands directly, and adding wireless peripherals or UIs. It is not meant to replace the main communication channels for long-range or high-volume data (e.g., it won’t stream video or large datasets well), but rather to complement them by offering convenience and redundancy in the immediate vicinity of the system.

5. Proposal: Bluetooth as a Redundant File Transfer Layer

One specific application of Bluetooth within MHP is as a **redundant layer for the current file transfer system**. We propose introducing a Bluetooth-based file transfer capability that can serve as a backup to the primary method (which may be Ethernet/Wi-Fi or physical media). This redundancy would enhance fault tolerance and provide an alternative path for data exchange when the primary link is unavailable or impractical. Below, we outline the benefits, challenges, and an implementation plan for this Bluetooth file transfer layer.

5.1 Benefits of a Bluetooth Redundant Channel

- **Fault Tolerance:** If the main file transfer mechanism fails (e.g., Wi-Fi network is down or cable disconnected), a Bluetooth link ensures that critical data (configuration files, small binary updates, log files, etc.) can still be exchanged locally. This is particularly useful for emergency situations or troubleshooting when other networks are unreliable. The robot could automatically fall back to Bluetooth to send an alert or receive a command file when it detects primary network loss.
- **Mobility and Convenience:** A Bluetooth link does not require existing infrastructure (like a router) or physical tethering. Team members can approach the robot with a laptop, tablet, or even a

smartphone and directly connect to it to transfer files. For example, updating a config script or retrieving a log bundle can be done on the spot via Bluetooth, without needing to plug in a USB drive or connect to a specific Wi-Fi SSID. This **ad hoc connectivity** is a big advantage in field tests or demos, where time and mobility are factors.

- **Isolation from Network Issues:** Bluetooth being a local point-to-point connection means it's immune to broader network outages or firewall issues. The file transfer over Bluetooth can be done even in secure facilities or isolated environments where the robot might not be allowed on the main network. It's a self-contained channel between the engineer's device and the robot.
- **Security (Local Scope):** While we must consider Bluetooth's own security (see Section 6), using a short-range direct link can be safer in some contexts than sending files over a larger network. For instance, if the team wants to update the robot without exposing it to the internet, a Bluetooth transfer from a nearby trusted device is an option. The limited range also means an attacker would need to be physically nearby to even attempt interference.
- **Independent Band:** Bluetooth operates on its own stack and frequencies; if the primary system is Wi-Fi at 5 GHz or an Ethernet connection, Bluetooth at 2.4 GHz provides a path that won't be affected by issues on those other channels (except general EMI). Even if the robot's main networking hardware fails, as long as power is on and the Bluetooth module is functional, some communication persists.

5.2 Challenges and Limitations

- **Throughput Limitations:** As discussed, Bluetooth (especially BLE) has much lower throughput than wired or Wi-Fi networks. Transferring large files (e.g. multi-megabyte logs or datasets) over Bluetooth will be **slow**. For example, a 10 MB file might transfer in a few seconds over Wi-Fi, but could take on the order of a few minutes via Classic Bluetooth, and even longer over BLE. This is a significant limitation for using Bluetooth as a file transfer medium ²³. The team must decide what file sizes and types are reasonable for the Bluetooth channel (e.g. perhaps limit it to critical configuration files, small patches, or summary logs, rather than full software images).
- **Range and Connectivity Constraints:** Bluetooth's short range means the transferring device must be near the robot. Unlike the primary file transfer which might work over a facility network or remotely, Bluetooth requires physical proximity. If the robot is inaccessible or in operation far from the operator, the Bluetooth backup channel won't help until someone can get close. This makes it a **local backup only**, not a remote failsafe.
- **Interference and Bandwidth Sharing:** The 2.4 GHz band can be crowded. If multiple Bluetooth devices are in use or heavy Wi-Fi interference is present, file transfer performance will degrade further. Additionally, Bluetooth is half-duplex and shares bandwidth among any other connected devices; if we have multiple Bluetooth links active (say, sensor streams plus a file transfer), they will contend for airtime and slow each other. We need to manage usage (perhaps pausing non-essential BT traffic during a file transfer).
- **Integration Complexity:** Implementing a robust file transfer over Bluetooth adds complexity to the system. We must handle the connection management, file segmentation, error recovery, and ensure

that using the Bluetooth channel doesn't negatively impact the robot's operations (e.g., saturating a CPU core with Bluetooth I/O). If using Classic Bluetooth, integrating the OBEX File Transfer Profile or Serial Port Profile on our systems may require additional libraries or services. If using BLE, we might have to design a custom GATT service for file transfer, which involves more development and testing. This is a non-trivial software task, though certainly feasible for the team.

- **Security Considerations:** Enabling file transfer means potentially exposing sensitive data or control files via a wireless channel. Without proper security, there is a risk (albeit small, given physical range limits) that an unauthorized device could connect and pull data or inject commands. We will address security in detail later, but it's a challenge to ensure that the convenience of Bluetooth doesn't open a backdoor. Strong authentication, encryption, and maybe user confirmation for certain operations would be needed.
- **Hardware Availability:** We need to ensure the robot's platform can support an additional Bluetooth channel without hardware conflicts. If the USB ports are limited or fully used, adding a dongle means freeing one up. If multiple systems (HostOS, SubOS layers in MHP) need Bluetooth access, we might have to share the adapter or have separate ones. Also, Bluetooth should be configured to start with the system, handle unexpected disconnects gracefully, etc., all of which requires careful setup.

5.3 Implementation Plan

To implement Bluetooth as a redundant file transfer layer in MHP, we propose the following steps:

1. **Hardware Setup:** Install a Bluetooth 5.x dual-mode adapter on the MHP main system (Huey's HostOS). For example, a USB Bluetooth dongle with an external antenna for better range. Ensure the driver (BlueZ on Linux SubOS or the Windows stack on HostOS, depending on where the file service will run) is properly configured. Verify that the adapter can operate in both Classic and BLE modes as needed.
2. **Protocol Selection:** Decide whether to use Classic Bluetooth or BLE for the file transfer service, or possibly support both for flexibility.
3. *Classic Option:* Use the **Serial Port Profile (SPP)** or the **Object Push/File Transfer Profile (OPP/FTP)**. SPP would create a virtual COM port over RFCOMM – we can then run a simple file transfer protocol (even as simple as XMODEM/ZMODEM or a custom framing) over this serial link. OPP/FTP would leverage existing Bluetooth profiles for file exchange (OBEX protocol), which might be integrated with OS tools (for instance, a laptop could just browse the robot's exposed Bluetooth filesystem). Classic will generally offer higher throughput than BLE and might be simpler to interface with PC tools.
4. *BLE Option:* Define a **GATT service** for file transfer. For example, create a service with characteristics for "file name", "file data chunk", "transfer control", etc. The laptop or phone app can connect as a BLE central, discover the service, and perform a sequence to upload/download a file in chunks (likely 20–512 bytes per chunk, depending on MTU negotiated). BLE would have the advantage of lower energy (not critical for a robot) and easier multi-platform support (since all modern phones support BLE well). However, achieving good throughput needs tuning (increase ATT MTU, use notifications for bursty data transfer).

5. *Hybrid*: Potentially support both by running dual services. For instance, advanced users with a laptop might use the Classic SPP-based transfer for speed, while a quick phone transfer might use BLE if an app is available. Initially, focusing on one method (perhaps Classic for simplicity) is advisable.
6. **Service Implementation**: Develop the **Bluetooth file transfer service** on the robot. If Classic SPP: run a background service that listens on an RFCOMM channel for a connection, then, once connected, negotiates a file transfer (this could be a simple custom protocol: e.g., the remote sends a command to either “SEND filename” or “RECV filename”, then streams bytes, with checksum or ACK for each block). If using OBEX FTP: set up a Bluetooth service record for OBEX and use an existing OBEX server library to handle file pushes/pulls to a designated directory on the robot. For BLE: implement the GATT server with the custom service and handlers for read/write on the characteristics. This step will likely involve low-level Bluetooth programming or using a framework like PyBluez (for Classic) or pygatt/nRF Connect SDK etc. (for BLE).
7. **Pairing & Authentication**: Establish how devices will be authorized to use this file service. Ideally, we restrict it to known trusted devices. Practically, this means the robot should either be paired with specific devices or use an authentication mechanism in the protocol. For Classic, pairing is required at the base level (and we can limit which devices know the PIN or accept only paired devices to connect). For BLE, we can require bonding and enforce encryption on the file transfer characteristics (i.e., make them readable/writable only when the connection is encrypted and authenticated). Use **LE Secure Connections** pairing by default for BLE ³² ³¹ and avoid “just works” pairing except in development. Possibly, have the robot announce a passkey on a screen or console for pairing to ensure MITM protection when pairing with a new client device.
8. **Integration with Existing System**: Modify the file management workflow in MHP to include Bluetooth as a fallback path. For example, if the primary file transfer is via a network share or web service, add logic such that if those fail, the system will log “Attempting Bluetooth transfer...” and wait for a Bluetooth connection. The team can initiate the Bluetooth transfer manually when needed (since Bluetooth won’t be constantly active sending files unless invoked). In practice, this might mean documenting a procedure: “If Wi-Fi transfer fails, connect via Bluetooth with our tool to retrieve the file.” Alternatively, more automation: the robot could detect lack of network and then advertise via BLE “I have log files ready” – an engineer’s device could pick that up and then connect. The exact integration depends on how autonomous we want it, but at minimum there should be a **clear operational mode** for Bluetooth transfer that the team can trigger.
9. **Testing and Performance Tuning**: Rigorously test the Bluetooth file transfer in various scenarios:
10. Measure throughput by sending files of known sizes and timing them at different ranges (close by vs. at 5m, etc.).
 11. Introduce interference (e.g. heavy Wi-Fi usage or multiple BLE devices) to see how it affects transfer reliability.
 12. Test boundary cases like dropping the connection mid-transfer (does our protocol resume or at least fail gracefully without corrupting data?).
 13. If BLE, test with different smartphone models and OSes if a mobile app is intended, ensuring broad compatibility.

14. Tune parameters: for BLE, use the largest ATT MTU both sides support and use data length extension if available to maximize payload per packet; adjust connection interval for a balance between speed and latency (fast for transfer, but perhaps slow it when idle). For Classic, ensure SPP buffer sizes are sufficient and perhaps increase the flush timeout if sending large bursts.
15. **Deployment and UX:** Develop any user-facing tools needed. For example, a simple PC utility or script to initiate the file transfer via Bluetooth could greatly simplify usage. On the robot's side, provide status feedback – e.g., log messages or an LED indicator when Bluetooth transfer is in progress. Train the team in how to pair with the robot and use the Bluetooth channel (this might involve distributing a pre-shared PIN or installing a companion app). It would be prudent to document the steps, like *“To retrieve logs via Bluetooth: put robot in maintenance mode, initiate pairing from your device with PIN ####, then run the `bt_pull_logs` script.”*

By following this implementation plan, the Monkey-Head Project will gain a robust secondary file transfer path. The solution leverages mature Bluetooth standards and should integrate with minimal hardware cost. We must remain mindful, however, that this channel is not intended for large data synchronization but rather as an **emergency and convenience link**. When used appropriately, it will significantly enhance the system's resilience and flexibility.

6. Security Considerations for Bluetooth Data Transmission

Security is paramount when introducing any wireless channel, and Bluetooth is no exception. While Bluetooth protocols have built-in security features, they also have known vulnerabilities if not configured properly. In the context of MHP, where potentially sensitive AI modules and control systems are at play, we need to address Bluetooth security on multiple levels: pairing security, link encryption, data access control, and operational policies.

Encryption and Pairing Security: Modern Bluetooth standards mandate strong encryption. BLE connections using LE Secure Connections employ 128-bit AES-CCM encryption for all traffic, with session keys derived via ECDH key exchange ³¹. This provides a high level of cryptographic security, comparable to internet protocols. Classic Bluetooth SSP (since v2.1) likewise uses ECDH (P-192 elliptic curve) to generate link keys and can achieve 128-bit equivalent security. The **important factor is the pairing method**: if “Just Works” pairing is used (no passkey or user confirmation), the link is encrypted but not authenticated – an attacker could theoretically perform a man-in-the-middle during the initial pairing. Therefore, for MHP: - Use authenticated pairing whenever possible. For example, require a passkey or numeric comparison for pairing any official laptops/phones with the robot. This might involve displaying a PIN on the robot's console or having a fixed passkey shared among the team. - If the robot has no interface to confirm a pairing code, a compromise is to use a pre-shared PIN code (something not trivial like “0000”) and compile it into the robot and the client tool – this at least keeps casual devices out. - **BLE Privacy Features:** BLE provides address randomization to prevent tracking ³³. The robot's BLE adapter should use a resolvable private address when advertising to avoid exposing a static MAC address to eavesdroppers. This doesn't directly secure data, but it helps with privacy. - After pairing (bonding), ensure the robot remembers bonded devices and will not accept new connections from unknown devices without pairing. This prevents an attacker from bypassing security by directly connecting if they somehow spoof the MAC address.

Access Control: Not all data should be freely accessible even to paired devices. We should enforce access control on the application level. For example, if using Bluetooth to fetch logs, perhaps require a command

from the user on the robot (or a physical key toggle) to enable that mode, so that even if someone paired, they can't continuously siphon data without notice. At minimum, log all Bluetooth connection events and what was transferred, so there's an audit trail.

Vulnerabilities and Updates: Be aware of known Bluetooth exploits: - **BlueBorne (2017):** a set of vulnerabilities that allowed malicious devices to take control without pairing. Patches have been released for most platforms. Ensure the robot's OS and Bluetooth stack are updated to include these patches. - **KNOB Attack (2019):** an attack on Classic Bluetooth's key negotiation, potentially downgrading the encryption key strength. The fix is to enforce not accepting extremely short encryption keys. Modern OS stacks have this fix – verify the dongle and stack are Bluetooth 5+ compliant which should include mitigation. - **BIAS (2020):** an impersonation attack where a previously bonded device could be impersonated. The countermeasure is also updated firmware that enforces stricter role switching checks. Again, using a fully updated Bluetooth 5.1+ controller should have this covered, but we should use latest firmware. - **Sniffing:** Bluetooth's encryption can be broken if the pairing was insecure. Using secure pairing prevents passive eavesdropping. However, one should operate under the assumption that a determined attacker with specialized hardware (Bluetooth sniffer arrays) could capture and possibly crack older pairing or any undisclosed flaws. Mitigate this by *limiting the sensitive data* sent over Bluetooth or adding an application layer encryption for extremely sensitive payloads (e.g., if transferring credentials or critical AI secrets, double-encrypt with AES-256 at the file level in addition to Bluetooth link encryption).

Operational Security: - Only keep Bluetooth discoverable when needed. The robot's Bluetooth should not be perpetually discoverable to everyone. We can have it in non-discoverable mode by default and only enter discoverable/pairing mode on a specific user action or during a maintenance window. This reduces the attack surface (an attacker won't even see the device unless they time it when we allow). - Implement timeouts for connections. If a device is paired but not actively doing anything, disconnect it after a period of inactivity. This minimizes the window for any hijacking or misuse. - For BLE, utilize the **lowest necessary privilege** on characteristics. If some characteristics (like a sensor reading) truly need to be accessible without encryption (not recommended, but for argument's sake), partition them so that critical controls or file transfers absolutely require encryption and perhaps an authentication token. BLE has attribute permissions; we should set all write operations and sensitive reads to require encryption & authentication. - Train team members to not pair random devices. They should know which devices are authorized. If the robot suddenly shows a pairing request from an unknown phone, they should reject it and investigate.

Testing Security: We should perform a mini **penetration test** for the Bluetooth interface. This includes: - Scanning for the device from an unauthorized phone (it should not be visible unless in pairing mode). - Trying a brute-force PIN pairing (the robot should have a limit or a long PIN to thwart this). - Using a Bluetooth sniffer during pairing to ensure our chosen method (numeric comparison or passkey) indeed prevents MITM (in practice, a MITM should not succeed if we do it right). - If possible, test the file transfer service for resilience (does it allow directory traversal? buffer overflow if sending a malformed file name? etc.). Treat it like any network service in terms of hardening.

Device Security: Ensure the Bluetooth module's firmware is updated. Many Bluetooth controllers can get firmware updates (for example, Intel or Broadcom Bluetooth chips on adapters). These updates often patch protocol vulnerabilities. Make checking for Bluetooth firmware updates part of the maintenance schedule.

By addressing the above points, the Monkey-Head Project can confidently utilize Bluetooth's convenience **without compromising security**. In summary, use strong pairing (LE Secure Connections / SSP with

authentication) ³⁰ ³¹ , keep the Bluetooth interface as low-profile as possible (non-discoverable except when needed), and stay updated on patches. With these measures, the risk of using Bluetooth for data transfer and control in MHP should remain low, and any potential attacker would face significant hurdles to exploit the system.

Conclusion

Integrating Bluetooth technology into the Monkey-Head Project stands to provide significant benefits in flexibility, redundancy, and user interaction. We have reviewed how Classic Bluetooth and BLE differ and where each might fit in MHP's architecture: Classic offers higher throughput for tasks like file transfer, whereas BLE provides low-power, low-latency links ideal for sensor data and control signals. Bluetooth's short-range capabilities (on the order of 10–50 m) align well with on-site operations and debugging, and its reliability can be maintained through careful attention to environmental factors and interference management.

We outlined various connectivity modes – from simple one-to-one pairings to complex mesh networks – concluding that while mesh networking is an impressive feature of BLE for IoT, the immediate needs of MHP can be met with point-to-point and star topologies (with the central robot communicating to multiple peripherals or user devices). Integration at the hardware level is straightforward using standard Bluetooth adapters, and the software stack (whether leveraging existing profiles or custom protocols) can be developed with readily available libraries.

Crucially, the proposal to add a Bluetooth-based redundant file transfer layer offers a new degree of fault tolerance. By enabling local wireless transfers of key data, the team can recover or update the system even when primary channels are down, all while untethered. The implementation plan provided gives a step-by-step guide to achieving this, from hardware selection to service development and testing.

Finally, we stressed that **security must be woven into every step** of Bluetooth adoption. When configured with modern security practices – authenticated pairing, encryption, limited discoverability – Bluetooth communication can be made robust against eavesdropping and unauthorized access ³² ³¹ . The team should remain vigilant with updates and best practices, treating the Bluetooth interface with the same care as any network interface.

In conclusion, adding Bluetooth to the Monkey-Head Project complements its adaptive, modular ethos. It will enable engineers and subsystems to interact with the system in new ways (like a “virtual tether” for diagnostics and control) and serve as a safety net in data delivery. With the information and plan detailed in this report, the team can proceed to prototype and integrate Bluetooth functionality, thereby enhancing the resilience and versatility of the MHP platform.

Sources:

1. MOKOSmart – “*Bluetooth vs Bluetooth Low Energy: A Detailed Comparison*” (2025) – Feature comparison of Classic vs BLE (data rates, latency, range, topology) ²⁰ ²⁵ ¹⁷ ⁴ ³⁷ .
2. Symmetry Electronics – “*Classic Bluetooth vs Bluetooth Low Energy – A Round by Round Battle*” – Practical throughput differences (~2 Mbps Classic vs ~0.1–0.25 Mbps BLE) and use-case considerations ²² .

3. Novel Bits – “*Bluetooth Low Energy: A Complete Guide*” – Explanation of BLE design goals, range factors (obstacles, antenna), and BLE data rates for Bluetooth 5 (2 M PHY, coded PHY) ¹⁶ ²⁶ .
4. Argenox – “*BLE Security & Privacy: A 2025 Guide*” – Highlights of modern BLE security (LE Secure Connections using ECDH, 128-bit AES-CCM encryption, pairing methods and best practices) ³² ³¹ .
5. Bluetooth SIG – *Bluetooth Core Specification & Mesh Profile 1.0* – (Referenced conceptually) details on piconet limits (7 devices) and BLE mesh capabilities (many-to-many device networking) ⁵ ⁴ .
6. Monkey-Head Project Documentation – MHP project README and design texts for context on system architecture and hardware (e.g. Huey’s Supermicro-based host) ³⁶ . (Internal project sources used for understanding integration context; no direct external citation available).

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ¹⁰ ¹¹ ¹⁴ ¹⁵ ¹⁷ ²⁰ ²¹ ²⁴ ²⁵ ³⁷ **Bluetooth VS Bluetooth Low Energy: A Detailed Comparison 2025**

<https://www.mokosmart.com/bluetooth-vs-bluetooth-low-energy-a-detailed-comparison/>

⁹ ¹² ¹⁶ ¹⁸ ¹⁹ ²⁶ ²⁷ ²⁹ **Bluetooth Low Energy (BLE): A Complete Guide**

<https://novelbits.io/bluetooth-low-energy-ble-complete-guide/>

¹³ ²² ²³ ²⁸ ³⁵ **Classic Bluetooth vs Bluetooth Low Energy - A Round by Round Battle | Symmetry Electronics**

<https://www.symmetryelectronics.com/blog/classic-bluetooth-vs-bluetooth-low-energy-a-round-by-round-battle/?srsltid=AfmBOop0-GfJMaKwvP5zDSQRPYjEWgV8CSFvisXuhqOYSS7KhgB8cFb>

³⁰ ³¹ ³² ³³ ³⁴ **Bluetooth Low Energy (BLE) Security & Privacy: A 2025 Guide**

<https://argenox.com/blog/bluetooth-low-energy-ble-security-privacy-a-2025-guide>

³⁶ **README.md**

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/35b9c528f1eabd6129f38135ef76b05c06bb71c2/README.md>